

Plataforma de Juegos Interactivos

Ahorcado & Pasapalabra · Java Swing · MVC · POO

25 min · 3 integrantes

Programación Orientada a Objetos



2 juegos
1 plataforma
3 capas MVC

Nombre del grupo · Curso 2025/2026

Plan de exposición

3 integrantes · ~8 minutos cada uno · 25 minutos en total

1	Integrante A Intro + Funcionalidades + Alcance + Arquitectura MVC	0–8 min
2	Integrante B Flujo de uso + Estructura código + Modelo dominio + POO (herencia, encaps., polimorfismo)	8–16 min
3	Integrante C Juegos (Ahorcado + Pasapalabra) + Persistencia + Demo en vivo + Conclusiones	16–25 min

 Objetivo:

que todos hablen
el mismo tiempo
y sin repetir.

¿Qué problema resuelve?

De una práctica de POO a una aplicación jugable con estructura real

Objetivo del proyecto

Crear una aplicación en Java donde un jugador pueda registrarse, iniciar sesión, elegir un juego, jugar partidas, guardar resultados y consultar estadísticas.

La mejora respecto a un ejercicio simple es que integra varias partes: interfaz, lógica de negocio y guardado de datos.

2

juegos

2

roles

3

capas MVC



Jugador

Se registra, inicia sesión, juega y consulta sus partidas.



Administrador

Consulta usuarios, últimas partidas y rankings por juego.

Funcionalidades principales

Lo que puede hacer el sistema desde el punto de vista del usuario



Registro / login

Validación básica de usuarios y contraseña; separación entre jugador y admin.



Menú de juegos

El usuario elige Ahorcado o Pasapalabra desde una interfaz Swing.



Partidas

Nueva partida, continuar partida en curso, pausar y finalizar.



Estadísticas

Historial y puntuaciones del jugador; rankings desde admin.



Administración

Consulta usuarios, últimas partidas y rankings por juego.

Alcance y resultado final

Qué se ha construido y qué queda fuera del proyecto

Incluye

- ✓ Interfaz gráfica Java Swing
- ✓ Modelo con usuarios, juegos y partidas
- ✓ Ahorcado y Pasapalabra funcionales
- ✓ Persistencia en ficheros TXT
- ✓ Javadocs generados en docs/

No incluye

- Base de datos SQL
- Juego online en red
- Sistema avanzado de seguridad
- Diseño profesional de videojuegos
- IA para preguntas dinámicas

Mensaje clave

El proyecto está pensado para demostrar organización, POO y separación de responsabilidades, con partidas individuales y datos persistidos en TXT.

Arquitectura MVC

Separación clara entre datos, interfaz y lógica de control



**La vista no debería decidir reglas de juego.
El modelo no debería conocer botones.
El controlador conecta ambos mundos.**

Estructura del código

Tres paquetes principales y responsabilidades diferenciadas

modelo/

- Usuario
- Jugador
- Admin
- Juego
- Ahorcado
- Pasapalabra
- Partida
- Pregunta
- SistemaJuegos

vista/

- VentanaPrincipal
- VentanaUsuario
- VentanaMenuJuego
- VentanaAhorcado
- VentanaPasapalabra
- VentanaEstadisticas
- VentanaAdmin

control/

- ControladorPrincipal
- ControladorUsuarios
- ControladorJuego
- ControladorAdmin
- ControladorFicheros

Flujo de uso de la aplicación

Desde el arranque hasta estadísticas o administración



Caso admin: si el usuario es administrador, el flujo abre funciones de gestión y ranking.

Modelo de dominio

Clases principales que representan el sistema



SistemaJuegos centraliza las listas de usuarios, juegos y partidas.

Herencia y abstracción

Clases base para evitar duplicar lógica común



Clase abstracta Juego

```
public abstract class Juego {  
    protected String nombre;  
  
    public Juego(String  
nombre) {  
        this.nombre = nombre;  
    }  
  
    public abstract Partida  
iniciarPartida(Jugador[]  
jugadores);  
}
```

Una clase abstracta define un contrato común: todas las subclases comparten una base, pero cada juego implementa su propia lógica.

Encapsulación y polimorfismo

Protección de datos y comportamiento dinámico



Encapsulación

Los atributos importantes son privados o protegidos. Se accede mediante métodos, evitando modificar el estado del objeto de cualquier forma.



Ejemplo

Partida controla id, juego, jugadores[], puntuaciones[], turno, fecha y estadoGuardado. En esta versión cada partida se crea con el jugador activo.



Polimorfismo

Una variable de tipo Juego puede apuntar a Ahorcado o Pasapalabra. El controlador llama a iniciarPartida(...) sin conocer todos los detalles internos.

Idea polimórfica

```
Juego juego = sj.buscarJuego(nombreJuego);  
Partida partida = juego.iniciarPartida(jugadores);
```

Resultado: menos duplicación, más orden y más facilidad para añadir otro juego en el futuro.

La clase Partida como núcleo

Une usuarios, juego, puntuación y estado guardado

Partida

- id
- juego
- jugadores[]
- puntuaciones[]
- fecha
- turnoActual
- estadoGuardado

Formato en partidas.txt

```
id;nombreJuego;fecha;finalizada;turno;  
user1,pts1;user2,pts2|estadoGuardado
```



Ahorcado

PALABRA;letrasUsadas;fallos



Pasapalabra

A=C;B=P;C=F;...

Esto permite pausar y continuar partidas porque el estado interno queda serializado como texto. Aunque Partida admite jugadores[], el flujo actual crea partidas individuales.

Juego 1: Ahorcado

Adivinar una palabra letra a letra con máximo de fallos

Mecánica

- Se elige una palabra aleatoria
- El jugador prueba letras
- Cada fallo aumenta el contador
- Máximo 6 fallos antes de perder
- La puntuación premia aciertos y resta fallos

EJEMPLO

_ A _ A

Letras usadas: A, E, R

Fallos: 2 / 6

Estado: JAVA;AER;2

calcularPuntuacion()

Juego 2: Pasapalabra

Rosco de preguntas sobre conceptos de programación

Rosco simplificado



Preguntas

Cada letra tiene un enunciado y una respuesta guardada en Pregunta.



Estados

C = correcta · F = fallada · P = pendiente/pasada. El jugador puede volver a las marcadas con P.



Puntuación

Puntuación y continuidad

Ambos juegos comparten la idea de partida pero no la misma lógica



Ahorcado

El resultado depende de letras acertadas, letras usadas y fallos acumulados.



Pasapalabra

El resultado depende de aciertos y errores en el roscó.



Partida

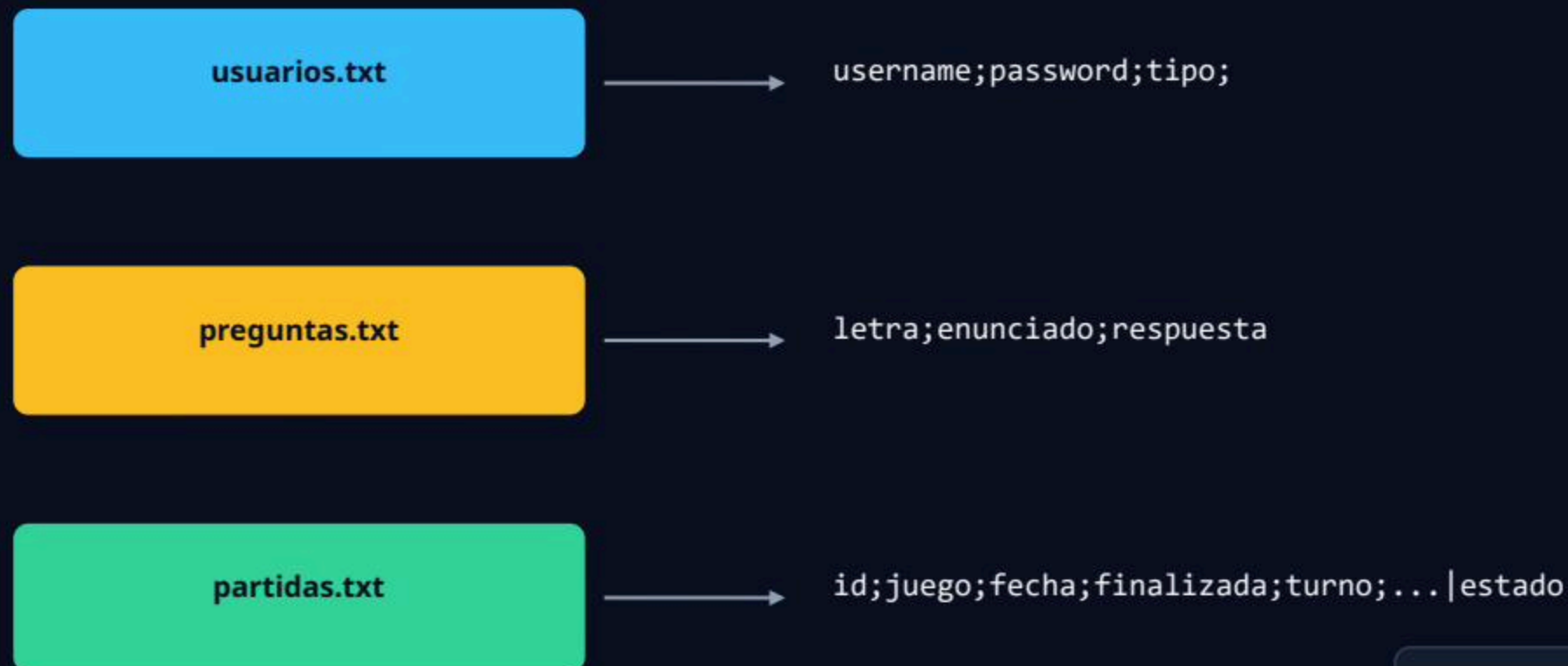
Guarda puntuación y estado para poder reanudar.

ControladorJuego decide:
iniciar · continuar · pausar · finalizar

El controlador no calcula todo directamente: delega en cada clase de juego y luego actualiza la partida.

Persistencia en ficheros TXT

Los datos sobreviven al cierre de la aplicación



ControladorFicheros

Lee, crea, añade, actualiza y guarda los datos en la carpeta data/.

Ventaja académica: es simple y visible. Limitación: no ofrece la robustez de una base de datos.

Estadísticas y administración

Información útil para jugador y administrador



Jugador

Consulta partidas propias, puntuaciones y partidas en curso.



Sistema

Filtra partidas por jugador, por juego o por estado.



Admin

Obtiene información global de usuarios y ranking por juego.



Datos

TXT



Filtros

SistemaJuegos



Ranking

ControladorAdmin

El administrador consulta datos guardados: usuarios, últimas partidas y rankings por juego.

Conclusiones y aprendizaje

Qué demuestra el proyecto y cómo podría evolucionar



Aprendizaje técnico

POO real: herencia, abstracción, encapsulación, polimorfismo y composición.



Aprendizaje de diseño

MVC ayuda a ordenar el código y a repartir responsabilidades.



Aprendizaje práctico

Persistencia, validaciones, interfaz Swing y coordinación entre clases.

Posibles mejoras futuras

- Migrar ficheros TXT a base de datos SQL
- Añadir más juegos reutilizando la clase Juego
- Mejorar seguridad de contraseñas
- Añadir pruebas automáticas y validaciones más completas

Cierre recomendado

"Hemos convertido los conceptos de POO en una aplicación completa, modular y jugable."

Demostración en vivo

Flujo completo · ~2–3 min · a cargo de Integrante C



Abrir proyecto en el IDE → ejecutar Main.java → seguir flujo: login → menú → partida → estadísticas

Duración estimada: 2–3 min · Integrante C maneja el teclado

Gracias por vuestra atención

¿Preguntas?

Java Swing · MVC · POO · Persistencia TXT · Ahorcado · Pasapalabra

Sugerencia para la defensa: abrir el proyecto y hacer una demo rápida de login → menú → partida → estadísticas.